

DOCUMENTACIÓN TÉCNICA

Sistema Automatizado de Gestión de Mano de Obra (SGMO)

1. Descripción General del Sistema

1.1 Contexto y Propósito

El **Sistema Automatizado de Gestión de Mano de Obra (SGMO)** nace de la necesidad de optimizar y auditar la clasificación de personal en contratos de obras e inspecciones técnicas (MOP / Asesorías).

El sistema automatiza la categorización de los trabajadores en tres niveles operacionales:

1. **Calificada** (Profesionales, Jefaturas, Prevencionistas).
2. **Semi Calificada** (Técnicos, Topógrafos, Operadores, Capataces).
3. **No Calificada** (Jornales, Paleteros, Bandereros, Ayudantes).

Su objetivo es eliminar el procesamiento manual de planillas, reduciendo errores humanos y agilizando la entrega de informes mensuales de dotación.

1.2 Arquitectura Modular

Para cumplir con los principios de diseño de software y mantener un código limpio (PEP 8), la aplicación se dividió en **5 módulos independientes**:

- **constantes.py**: Define los datos inmutables del sistema y listas/conjuntos de palabras clave.
- **validador.py**: Contiene las funciones de validación de entrada de datos (RUT, campos obligatorios).
- **clasificador.py**: Contiene la lógica de negocio para clasificar trabajadores y la función recursiva de conteo.
- **gestor_datos.py**: Encargado de la creación de estructuras de datos y la lectura/escritura de archivos CSV usando rutas dinámicas (`os`).
- **main.py**: Punto de entrada del programa. Coordina la interfaz por terminal y el menú de navegación con el usuario.

2. Estructuras de Datos Utilizadas

En este proyecto se aplicaron de forma justificada las cuatro estructuras de datos principales de Python:

Estructura	Ejemplo en el Código	Justificación Técnica
Listas (list)	<pre>trabajadores = []</pre>	Se utiliza para almacenar el conjunto ordenado y dinámico de todos los trabajadores procesados. Permite agregar (append), iterar y exportar los datos.
Diccionarios (dict)	<pre>{"rut": ..., "nombre": ..., "cargo": ..., "clasificacion": ...}</pre>	Utilizado para representar la ficha o perfil individual de cada trabajador mediante pares clave-valor, facilitando el acceso a sus propiedades.
Tuplas (tuple)	<pre>CATEGORIAS = ("CALIFICADA", "SEMI CALIFICADA", "NO CALIFICADA")</pre>	Garantiza la inmutabilidad de las categorías oficiales exigidas. Evita modificaciones accidentales durante la ejecución del programa.
Conjuntos (set)	<pre>ruts_procesados = set() PALABRAS_CALIFICADAS = {...}</pre>	Aprovecha la propiedad de elementos únicos para evitar el registro duplicado de RUTs y realizar búsquedas de palabras clave con complejidad de tiempo óptima $O(1)$.

3. Funcionalidades Implementadas

3.1 Entrada, Salida y Validación de Datos

- **Captura y Visualización:** Interacción fluida mediante la terminal utilizando `print()` y `f-strings` para el formateo alineado de tablas e informes.
- **Manejo de Excepciones y Rutas:** Implementación del módulo `os` para resolver la ruta absoluta del archivo CSV, garantizando su lectura sin importar la carpeta desde la que se ejecute la terminal.
- **Validación de Entradas:** Uso de funciones en `validador.py` para verificar el formato de RUT y prevenir el ingreso de cadenas vacías.

3.2 Estructuras de Control de Flujo

- **Condicionales (`if / elif / else`):** Implementados en `clasificar_trabajador()` para determinar automáticamente el nivel del trabajador analizando si posee título profesional/técnico o buscando palabras clave en su cargo.
- **Bucles (`while` y `for`):** * Ciclos `while` en el menú principal (`main.py`) para mantener la aplicación activa hasta que el usuario decida salir.
 - Ciclos `for` para iterar listas de trabajadores al momento de visualizarlos en pantalla o escribirlos en el archivo CSV.
- **Control de Saltos (`break` y `continue`):** Usados para la navegación limpia del menú y para ignorar registros duplicados o vacíos en el procesamiento.

3.3 Funciones y Recursividad

- **Modularidad:** Funciones con responsabilidad única, parámetros claros y retornos explícitos (`return`).
- **Función Recursiva:** Se implementó la función `contar_trabajadores_recursivo()` en `clasificador.py`. Esta función realiza el conteo total acumulado de la lista de trabajadores mediante llamadas recursivas sobre el índice de la lista, cumpliendo con la evaluación teórica de patrones recursivos.

3.4 Persistencia de Datos (Archivos CSV)

- **Importación:** Carga de nóminas externas mediante `csv.DictReader`, clasificando automáticamente a cada trabajador durante la lectura.
- **Exportación:** Generación del informe consolidado final utilizando `csv.DictWriter`, almacenando las columnas estandarizadas en un nuevo archivo físico `.csv`.

4. Conclusiones y Cumplimiento de Estándares (PEP 8)

- **Legibilidad:** Código documentado mediante *docstrings* en cada módulo y función.

- **Mantenibilidad:** La separación modular permite que en el futuro la lógica pueda conectarse fácilmente a una base de datos o una interfaz web (Django) sin rehacer la lógica del clasificador.
- **Performance:** La selección estratégica de conjuntos (**set**) e iteraciones controladas evita cuellos de botella al procesar listas masivas de trabajadores.